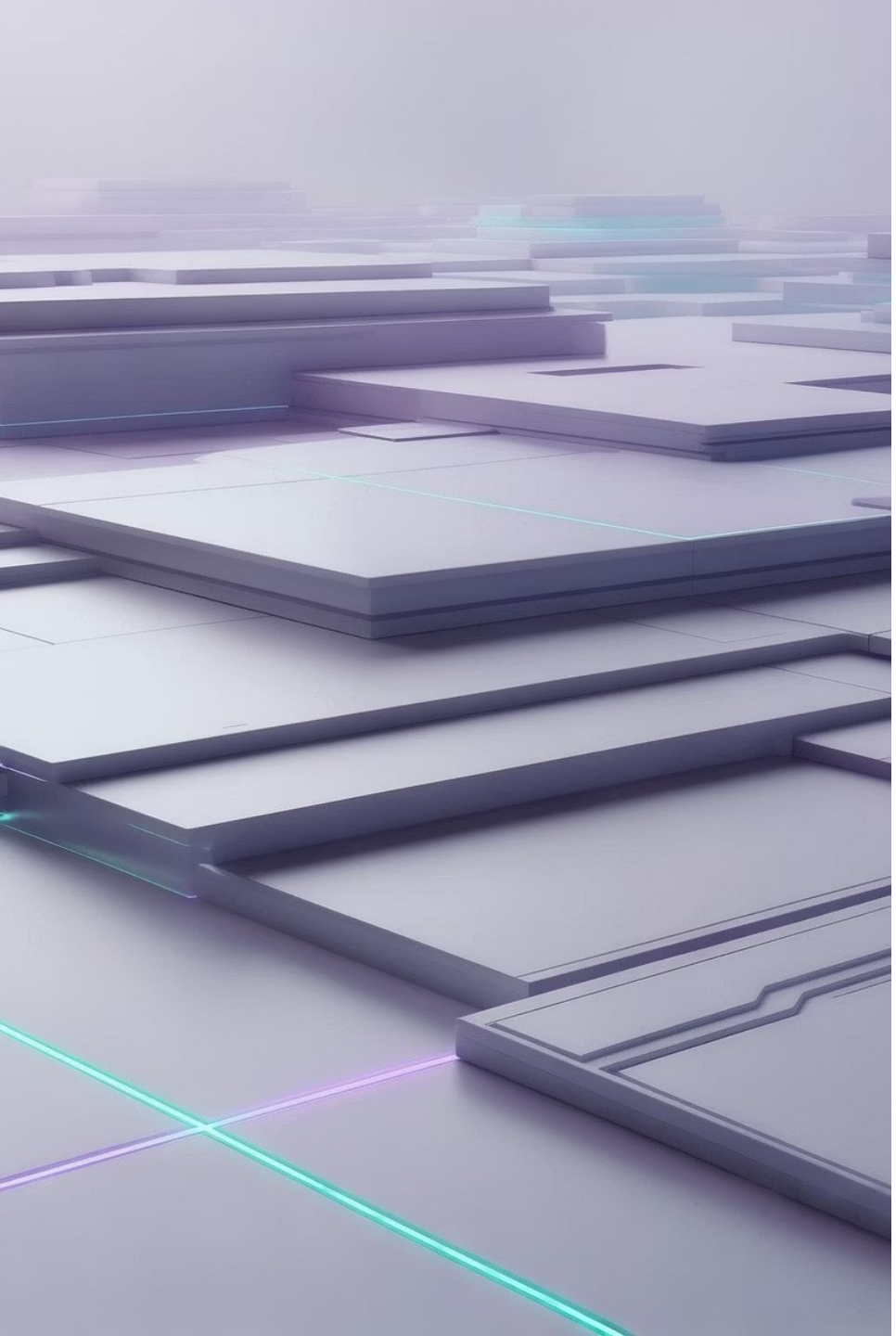


WEB-ENGINEERING 1 · TERMIN 5 / 9

CLIENT, SERVER & HTTP

Requests, Statuscodes – und warum es live anders ist

TAISC The AI Software Company

Joshua Heller · 27.10.2026

RÜCKBLICK

Wer zeigt seine Hausaufgabe?

Zwei Freiwillige, je vier Minuten – dafür gibt es einen Bonuspunkt.

Freiwillig

Niemand muss. Wer zeigt, bekommt +1 Punkt fürs Team (max. +5 im Semester).

Halb fertig reicht

Keine Musterschüler nötig – gerade an Baustellen lernen alle am meisten.

Hängen geblieben?

Auch das zählt: Zeigt, wo es nicht weiterging – wir lösen es gemeinsam.

Letzte Woche

Layout mit Flexbox und Grid, eine Seite auf Tailwind umgestellt.

© 2026 TAISC – The AI Software Company GmbH

KURZ ZURÜCK

Das war letzte Woche, das kommt heute

Letzte Woche

Das solltet ihr mitgenommen haben

- Flexbox für eine Richtung, Grid für zwei
- Mobile first: klein anfangen, dann erweitern
- Tailwind ist CSS mit kurzen Namen
- npm, package.json und der Build mit Vite

Heute

Darauf bauen wir auf

- Was zwischen Browser und Server passiert
- HTTP: Methoden, Statuscodes, Header
- Der Network-Tab als wichtigstes Werkzeug
- Euer Projekt geht heute online

© 2026 TAISC – The AI Software Company GmbH

WO WIR STEHEN

Euer Projekt wächst mit jedem Termin

Heute kommt dazu: Deployed

06.10.
HTML-Gerüst
Seiten, Navigation, Formular

13.10.
Eigenes CSS
Farben, Schrift, Abstände

20.10.
Layout + Tailwind
Flex, Grid, responsive

27.10.
Deployed
Live-URL auf GitHub Pages

17.11.
Interaktiv
JavaScript, Events, Zustand

24.11.
Echte Daten
fetch aus einer öffentlichen API

01.12.
Feinschliff
Komponenten, Bonus, README

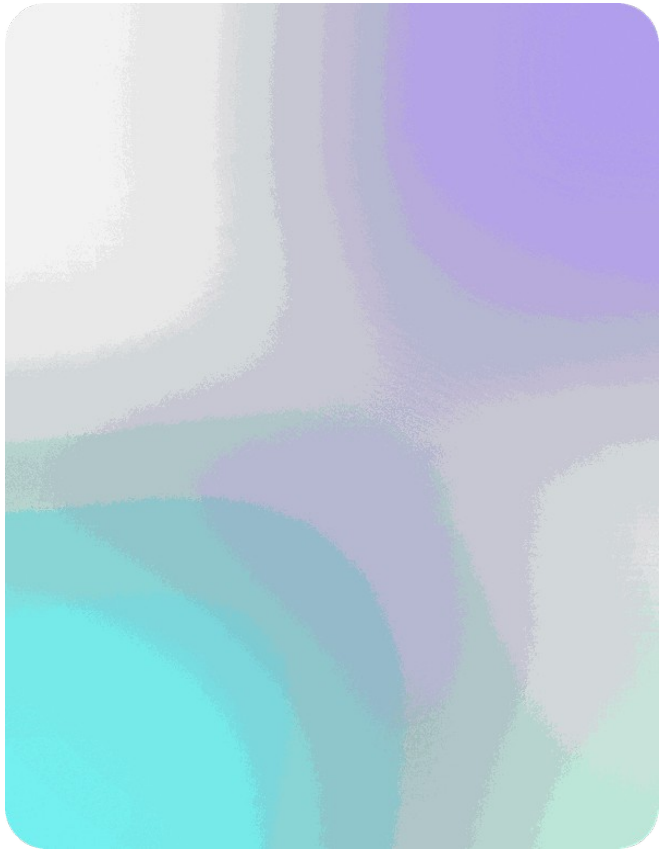
08.12.
Präsentation
Demo + Rückfragen

© 2026 TAISC – The AI Software Company GmbH

KAPITEL 1

Client und Server

Wer fragt, wer antwortet?



ERINNERUNG

Der Weg einer Anfrage

CLIENT

Browser

Chrome, Safari, Firefox

```
https://dhbw.de/
```

1. **DNS:** Wo wohnt dhbw.de? → 141.72.x.x
2. `GET / HTTP/2 →`
3. `← 200 OK + index.html`
4. Browser liest HTML, lädt CSS, JS, Bilder nach
5. **Rendering:** DOM + CSS → Pixel auf dem Bildschirm

SERVER

Webserver

liefert Dateien oder erzeugt sie

```
index.html
style.css
app.js
```

Heute zoomen wir in die Schritte 1 bis 3.

© 2026 TAISC – The AI Software Company GmbH

ROLLEN

Client und Server sind Rollen, keine Geräte

Browser → Webserver

Browser ⇄ Webserver

Client: euer Browser
Server: dhbw.de
Ihr ruft eine Seite auf.

App → API

App ⇄ API

Client: die DB-Navigator-App
Server: die Fahrplan-API
Die App holt JSON, kein HTML.

Server → anderer Server

Node-Server ⇄ Wetter-API

Client: euer Node-Server
Server: die Wetter-API
Ein Server kann selbst Client sein.

Euer Rechner → euer Rechner

Browser ⇄ Vite

Client: Browser
Server: Vite auf localhost:5173
Beide Rollen auf einem Gerät.

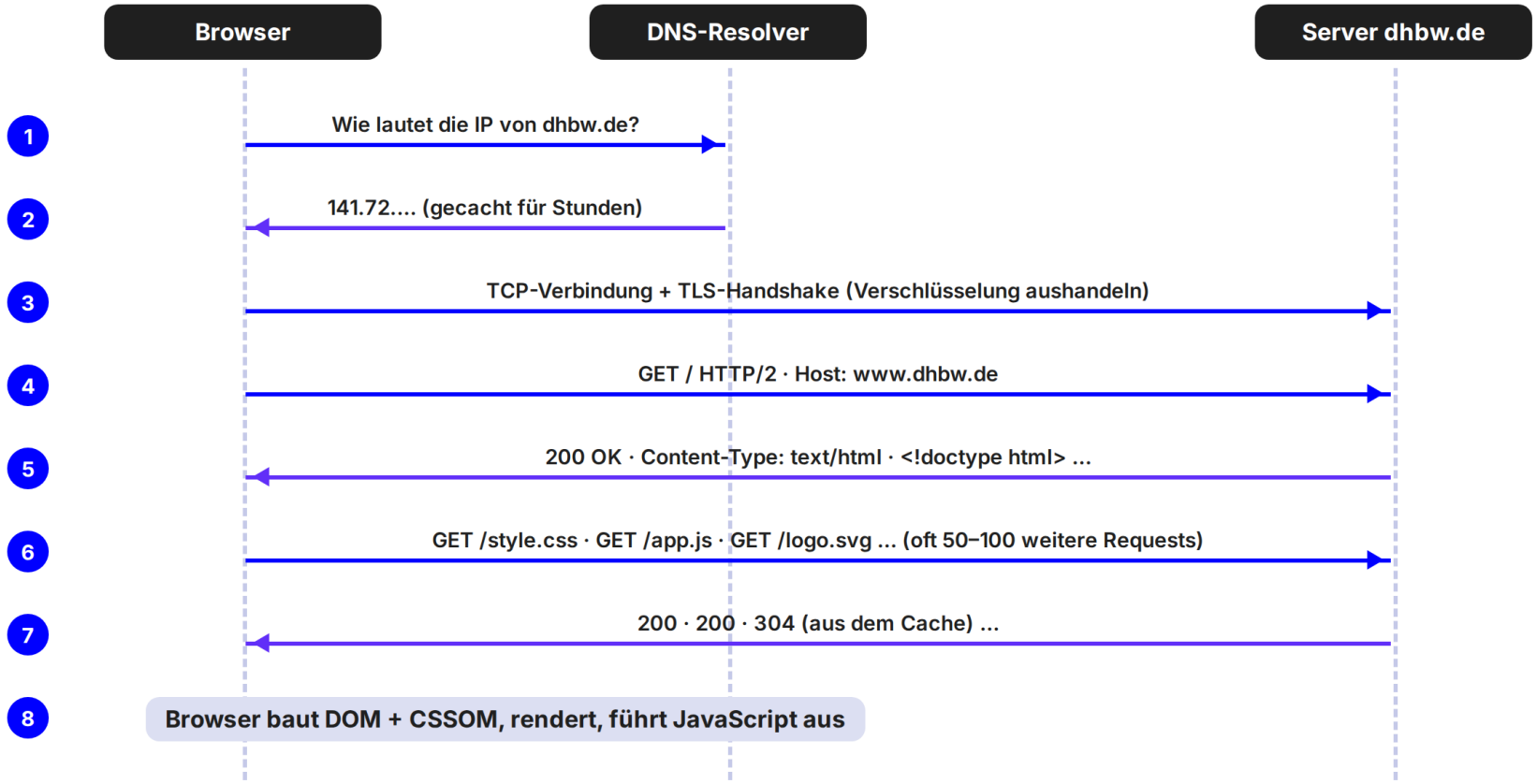
Client und Server sind Rollen, keine Geräte: Wer fragt, ist Client. Wer antwortet, ist Server.

Sobald ein Programm etwas anfragt, ist es Client. Sobald es antwortet, ist es Server.

© 2026 TAISC – The AI Software Company GmbH

SCHRITT FÜR SCHRITT

Was passiert, wenn ihr dhbw.de eintippt?



© 2026 TAISC – The AI Software Company GmbH

URL

Eine Adresse, sechs Teile

```
https://api.example.de:443/v1/abfahrten?halt=Kronenplatz&n=5#top
```

Protokoll
Wie wird gesprochen? https = verschlüsselt

Host
Welcher Rechner? Per DNS → IP

Port
Welche Tür? Bei https meist weggelassen

Pfad
Welche Ressource auf dem Server?

Query
Zusatzangaben: Name=Wert, mit & getrennt

Fragment
Sprungmarke – geht nie an den Server

Das Fragment (#top) bleibt im Browser. Alles davor geht an den Server.

© 2026 TAISC – The AI Software Company GmbH

URLS LESEN

Vier echte Beispiele

Google-Suche

`https://www.google.com/search?q=dhbw+karlsruhe&hl=de`

q = Suchbegriff (Leerzeichen als +), hl = Sprache der Oberfläche

YouTube an Sekunde 42

`https://www.youtube.com/watch?v=abc123XYZ&t=42`

v = Video-ID, t = Startzeit in Sekunden – deshalb funktionieren geteilte Links mit Zeitstempel

GitHub: ein Ordner im Kurs-Repo

`https://github.com/joshuaheller/dhbw-web-engineering-1/tree/main/bausteine`

Nur Pfad, keine Query: Konto / Repo / Branch / Ordner – der Pfad ist hier selbst die Information

Umlaute und Leerzeichen

`…/suche?halt=M%C3%BChlburger%20Tor`

„Mühlburger Tor" kodiert: ü → %C3%BC, Leerzeichen → %20. Das macht der Browser automatisch.

© 2026 TAISC – The AI Software Company GmbH

DARUNTER

DNS, IP und Ports

DNS

Das Telefonbuch des Internets

- dhbw.de → 141.72.x.x
- Wird gecacht – Änderungen dauern
- Probiert es: nslookup dhbw.de

IP-Adresse

Die Hausnummer

- IPv4: 141.72.10.1
- IPv6: 2001:7c0:...
- 127.0.0.1 = localhost = ihr selbst

Port

Die Wohnungstür

- 443 für HTTPS, 80 für HTTP
- 5173 für Vite, 3000 für viele Dev-Server
- Ein Port, ein Programm

Darunter liegt TCP bzw. bei HTTP/3 QUIC – Details in Netztechnik.

© 2026 TAISC – The AI Software Company GmbH

DNS

Vom Namen zur IP-Adresse

1 · Browser-Cache
Schon mal gefragt?

→

2 · Betriebssystem
hosts-Datei, OS-Cache

→

3 · Resolver
vom Router / Provider oder 1.1.1.1

→

4 · Fragt sich durch
Root-Server → „.de"-Server (DENIC) → Nameserver von dhbw.de

Antwort: www.dhbw.de → IP-Adresse · wird gecacht (TTL), deshalb dauern DNS-Änderungen manchmal Stunden

Erst wenn die IP bekannt ist, kann der Browser überhaupt eine Verbindung aufbauen.

© 2026 TAISC – The AI Software Company GmbH

PORTS

Ein Rechner, viele Türen

Ein Rechner
`IP 192.168.1.20`

`:443`	Webserver mit HTTPS
`:5173`	Vite Dev-Server
`:3000`	euer Node-Server
`:5432`	PostgreSQL-Datenbank

IP = Hausadresse, Port = Wohnungstür. Pro Port lauscht genau ein Programm – sonst: `EADDRINUSE`

© 2026 TAISC – The AI Software Company GmbH

SELBST AUSPROBIEREN

Drei Befehle fürs Terminal

Terminal (macOS, Linux, Windows)

```
# 1. Welche IP hat dhbw.de?
nslookup www.dhbw.de

# 2. Ist der Rechner erreichbar – und wie schnell?
ping -c 3 www.dhbw.de

# 3. Nur die Antwort-Header holen (-I)
curl -I https://www.dhbw.de

# 4. Wer hört auf welchem Port? (macOS/Linux)
lsof -i :5173
```

Was ihr seht

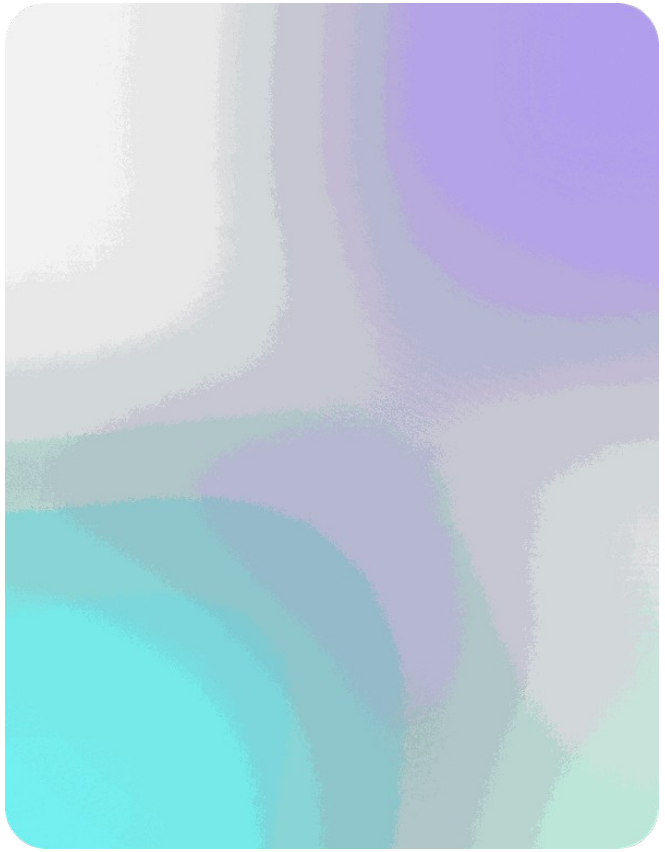
- nslookup: die IP-Adresse hinter dem Namen
- ping: Antwortzeit in Millisekunden (manche Server antworten absichtlich nicht)
- curl -I: Statuscode und Header – ohne HTML
- lsof: welches Programm Port 5173 belegt, wenn Vite „Port in use“ meldet

© 2026 TAISC – The AI Software Company GmbH

KAPITEL 2

HTTP

Die Sprache zwischen Browser und Server.



ANATOMIE

Request und Response

REQUEST (BROWSER → SERVER)

```
GET /api/abfahrten?halt=Kronenplatz HTTP/2
Host: api.example.de
Accept: application/json
Authorization: Bearer eyJhbGci…

(Body leer bei GET)
```

Methode · Pfad · Header · Body

RESPONSE (SERVER → BROWSER)

```
HTTP/2 200 OK
Content-Type: application/json
Cache-Control: max-age=30

[{"linie":"S1","ziel":"Bad Herrenalb",
  "in":4}]
```

Statuscode · Header · Body

© 2026 TAISC – The AI Software Company GmbH

METHODEN

Was der Client will

Methode	Zweck	Body	Beispiel
GET	Daten lesen	nein	GET /abfahrten?halt=Kronenplatz
POST	Neu anlegen	ja	POST /favoriten
PUT	Komplett ersetzen	ja	PUT /favoriten/7
PATCH	Teilweise ändern	ja	PATCH /favoriten/7
DELETE	Löschen	selten	DELETE /favoriten/7
OPTIONS	Was ist erlaubt? (CORS)	nein	automatisch vom Browser

© 2026 TAISC – The AI Software Company GmbH

METHODEN IN DER PRAXIS

Eine Favoriten-API, sechs Anfragen

Methode	Pfad	Was passiert	Antwort
GET	/favoriten	Liste aller Favoriten holen	200 + JSON
GET	/favoriten/7	Einen Favoriten holen	200 / 404
POST	/favoriten	Neuen Favoriten anlegen (Daten im Body)	201 Created
PATCH	/favoriten/7	Einen Favoriten teilweise ändern, z. B. umbenennen	200
PUT	/favoriten/7	Favoriten komplett ersetzen	200
DELETE	/favoriten/7	Favoriten löschen	204 No Content

Der Pfad nennt das **Ding** (Substantiv), die Methode die **Tätigkeit** (Verb). So sind die meisten APIs aufgebaut („REST“).

© 2026 TAISC – The AI Software Company GmbH

STATUSCODES

Was der Server antwortet

Code	Bedeutung	Typischer Grund
200	OK	Alles gut
201	Created	Nach erfolgreichem POST
301 / 304	Moved / Not Modified	Umleitung bzw. Cache ist noch gültig
400	Bad Request	Anfrage kaputt, z. B. falsches JSON
401 / 403	Unauthorized / Forbidden	Nicht angemeldet / keine Rechte
404	Not Found	Falscher Pfad – Klassiker nach dem Deployen
429	Too Many Requests	Rate-Limit der API erreicht
500	Internal Server Error	Fehler im Server-Code
502 / 503	Bad Gateway / Unavailable	Server dahinter down oder überlastet

Merkregel: 2xx gut, 3xx woanders, 4xx du bist schuld, 5xx der Server ist schuld.

© 2026 TAISC – The AI Software Company GmbH

STATUSCODES IN DER PRAXIS

Die Codes, die ihr wirklich seht

200 OK
Seite oder Daten kommen an. Der Normalfall.

201 Created
Formular gespeichert, neuer Eintrag angelegt.

301 Moved
http:// wird dauerhaft auf https:// umgeleitet.

304 Not Modified
„Hast du schon" – Browser nimmt die Datei aus dem Cache.

400 Bad Request
Kaputtes JSON oder Pflichtfeld fehlt.

401 Unauthorized
Nicht eingeloggt oder API-Key fehlt.

403 Forbidden
Eingeloggt, aber keine Berechtigung.

404 Not Found
Tippfehler im Pfad – der Klassiker nach dem Deployen.

429 Too Many
API-Limit erreicht, später nochmal.

500 Internal Error
Der Code auf dem Server ist abgestürzt.

502 / 504
Ein Server dahinter antwortet nicht (Gateway).

418 I'm a teapot
Ein Aprilscherz von 1998 – steht wirklich in einem RFC.

Grün: alles gut · Blau: woanders/Cache · Orange: der Fehler liegt beim Client · Rot: beim Server

© 2026 TAISC – The AI Software Company GmbH

HEADER

Metadaten zu jeder Nachricht

Content-Type

Was im Body steckt: text/html, application/json, image/png.

Authorization

Wer fragt. Meist Bearer-Token oder API-Key.

Cache-Control

Wie lange darf der Browser die Antwort wiederverwenden?

Cookie / Set-Cookie

Kleine Datenstücke, die der Browser mitschickt – z. B. Login.

User-Agent

Welcher Browser fragt. Wird gern zum Blockieren von Bots genutzt.

Access-Control-Allow-Origin

Darf eine andere Domain diese Antwort lesen? Stichwort CORS.

© 2026 TAISC – The AI Software Company GmbH

CACHING

Warum eure Änderung nicht ankommt

Wer speichert

An vier Stellen

- Browser-Cache auf eurem Rechner
- CDN zwischen euch und dem Server
- Der Server selbst
- Der Service Worker, falls vorhanden

Wie gesteuert wird

Header

- Cache-Control: max-age=3600
- ETag: Version der Datei
- 304 Not Modified: nichts Neues

Was ihr tut

Wenn es klemmt

- DevTools → Network → Disable cache
- Hartes Neuladen: Cmd/Strg + Shift + R
- Im Build bekommen Dateien einen Hash im Namen

„Bei mir sieht es noch alt aus“ ist in neun von zehn Fällen der Cache.

© 2026 TAISC – The AI Software Company GmbH

CACHING IN DREI SCHRITTEN

Wie der Browser Dateien wiederverwendet

1 · Erster Besuch

```
GET /style.css
← 200 OK
  Cache-Control: max-age=60
  ETag: "v17"
```

Datei kommt komplett, Browser merkt sich Version „v17“.

2 · Nach 5 Minuten

```
GET /style.css
  If-None-Match: "v17"
← 304 Not Modified
```

„Hab ich noch – ist sie noch aktuell?“ Server: ja. Kein Download.

3 · Mit Vite-Build

```
index-BYMXB0qK.css
→ ändert sich der Inhalt,
  ändert sich der Name:
index-K93fQx2a.css
```

Darf ewig gecacht werden. Neue Version = neuer Name = neuer Download.

© 2026 TAISC – The AI Software Company GmbH

VERSIONEN

HTTP/1.1, HTTP/2, HTTP/3

HTTP/1.1

1997

- Text-basiert, lesbar
- Eine Anfrage nach der anderen pro Verbindung

HTTP/2

2015

- Binär, viele Anfragen parallel
- Header komprimiert
- Heute Standard

HTTP/3

2022

- Läuft über QUIC statt TCP
- Schneller bei schlechtem Netz
- Wird automatisch genutzt

HTTPS

immer

- HTTP + TLS-Verschlüsselung
- Ohne HTTPS keine Kamera, kein GPS, kein Service Worker

© 2026 TAISC – The AI Software Company GmbH

HTTPS

Was das Schloss bedeutet – und was nicht

Verschlüsselt

Niemand im WLAN, beim Provider oder unterwegs kann mitlesen oder Inhalte verändern.

Zertifikat

Beweist: Dieser Server gehört wirklich zu dhbw.de. Ausgestellt von einer Zertifizierungsstelle, z. B. Let's Encrypt (kostenlos).

Mixed Content

Eine https-Seite lädt kein `http://`-Bild oder -Script. Häufiger Fehler nach dem Deployen.

Das Schloss heißt nicht „seriös" – auch Phishing-Seiten haben HTTPS. Es heißt nur: Die Verbindung ist sicher.

© 2026 TAISC – The AI Software Company GmbH

LIVE

HTTP ohne Browser

Terminal

```
curl -i https://api.github.com/users/octocat

HTTP/2 200
content-type: application/json
x-ratelimit-remaining: 59

{
  "login": "octocat",
  "public_repos": 8,
  ...
}
```

- curl zeigt, was der Browser sonst versteckt
- -i gibt die Header mit aus
- So testet ihr APIs, bevor ihr eine Zeile JavaScript schreibt

© 2026 TAISC – The AI Software Company GmbH

JSON

Die Sprache der APIs

Antwort von api.github.com/users/octocat (gekürzt)

```
{
  "login": "octocat",          ← Text (String)
  "public_repos": 8,           ← Zahl
  "site_admin": false,         ← true / false
  "bio": null,                 ← „nichts“
  "plan": {                    ← Objekt im Objekt
    "name": "free"
  },
  "tags": ["git", "web"]       ← Liste (Array)
}
```

Sechs Bausteine, sonst nichts

- Schlüssel immer in doppelten Anführungszeichen
- Kein Komma nach dem letzten Eintrag
- Keine Kommentare – die Pfeile hier sind nur zur Erklärung
- Im Browser: Network → Response oder Preview
- In JavaScript: response.json() macht daraus ein Objekt

© 2026 TAISC – The AI Software Company GmbH

DEVTOOLS

Der Network-Tab

Ab heute schaut ihr hier zuerst, wenn etwas nicht lädt.

Status

Rote Zeilen sind 4xx oder 5xx. Da fängt die Suche an.

Headers

Request- und Response-Header, auch die für CORS.

Response / Preview

Was wirklich zurückkam – oft nicht das, was ihr erwartet.

Timing

Wasserfall: Was bremst die Seite?

Throttling

Die Seite mit langsamem 3G testen.

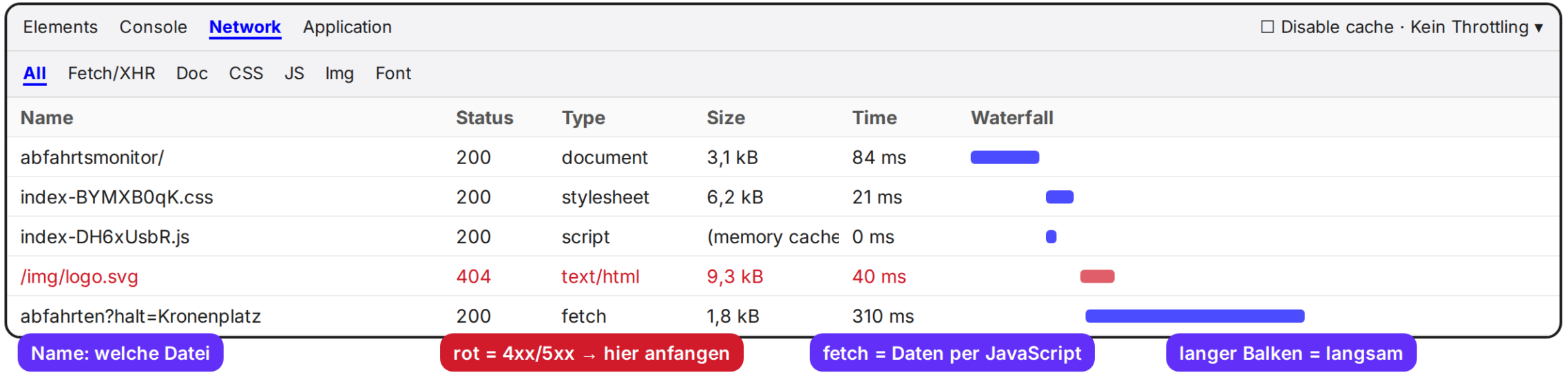
Disable cache

Wenn Änderungen „nicht ankommen“, liegt es oft am Cache.

© 2026 TAISC – The AI Software Company GmbH

DEVTOOLS IN BILDERN

So lest ihr den Network-Tab



Nachgebaut am Beispiel eures Projekts: das Logo fehlt (404), die API antwortet langsam (310 ms).

© 2026 TAISC – The AI Software Company GmbH

GLEICH LIVE

Drei Seiten, drei Lektionen

Network-Tab auf, Seite neu laden, erste Zeile anklicken.

wikipedia.org

Schlank und klassisch: ein HTML-Dokument, ein paar CSS- und Bilddateien. So sah das Web lange aus.

tagesschau.de

Der Inhalt kommt per JSON nachgeladen. Filter auf Fetch/XHR stellen – dort seht ihr die eigentliche API.

Eure eigene Seite

Gleich in der Werkstatt: lokal und deployed vergleichen. Welche Requests fehlen live?

Filter, Statusspalte, Preview – drei Klicks, die euch stundenlange Fehlersuche sparen.

© 2026 TAISC – The AI Software Company GmbH

CORS

Warum fetch manchmal rot wird

Eure Seite

team.github.io

```
fetch("https://api.
  wetter.example/heute")
```

→

Request mit
Origin: team.github.io

Fremde API

api.wetter.example

Antwortet – und entscheidet per Header, wer lesen darf.

Mit Header

`Access-Control-Allow-Origin: *`

→ Browser gibt die Daten an euer JavaScript

Ohne Header

`Access to fetch … has been blocked by CORS policy`

→ Antwort kam an, aber der Browser hält sie zurück

CORS ist eine Regel des Browsers, nicht des Servers. curl und Node kennen sie nicht – deshalb klappt es dort.

© 2026 TAISC – The AI Software Company GmbH

CORS BEHEBEN

Warum es CORS gibt – und was ihr tun könnt

Warum?

Schutz eurer Logins

- Ohne CORS könnte jede fremde Seite im Hintergrund mit euren Cookies eure Mails oder euer Konto abfragen
- Der Browser gibt Antworten nur an Seiten weiter, die der Server ausdrücklich erlaubt

Weg 1

API mit CORS wählen

- Die meisten öffentlichen APIs erlauben es: PokéAPI, Open-Meteo, Open Food Facts
- Vorher testen: fetch in der Console der eigenen Seite

Weg 2

Eigene API: Header setzen

- Access-Control-Allow-Origin: * oder eure Domain
- So im Baustein t5-mini-server

Weg 3

Eigener Server dazwischen

- Server fragt Server – dort gibt es kein CORS
- Mit Vercel, Netlify oder Cloudflare als Serverless Function

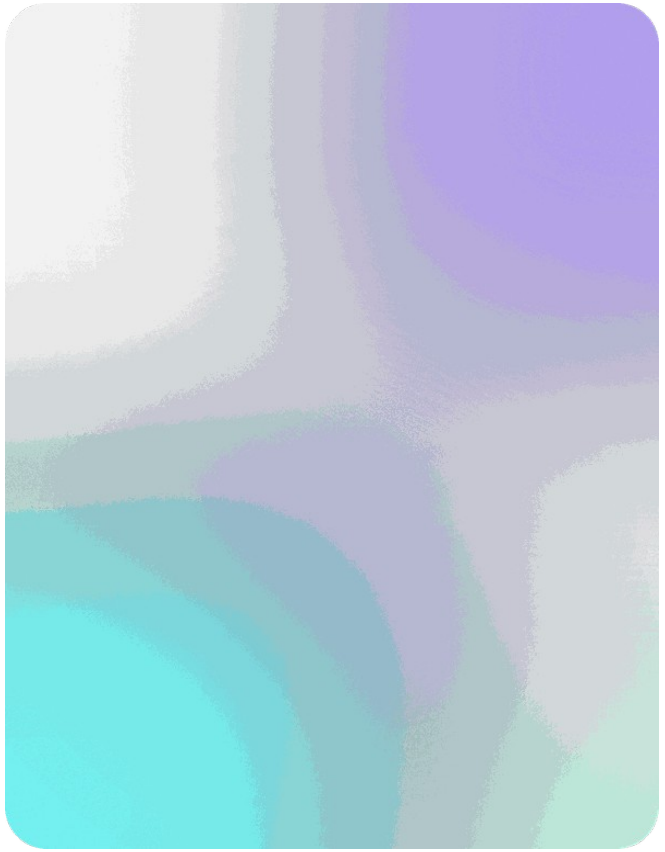
Im Frontend lässt sich CORS nicht abschalten. Die Erlaubnis kommt immer vom Server.

© 2026 TAISC – The AI Software Company GmbH

KAPITEL 3

„Läuft bei mir“

Datei, lokaler Server, Hosting.



KENNT IHR DAS?

Der häufigste Satz in Softwareprojekten



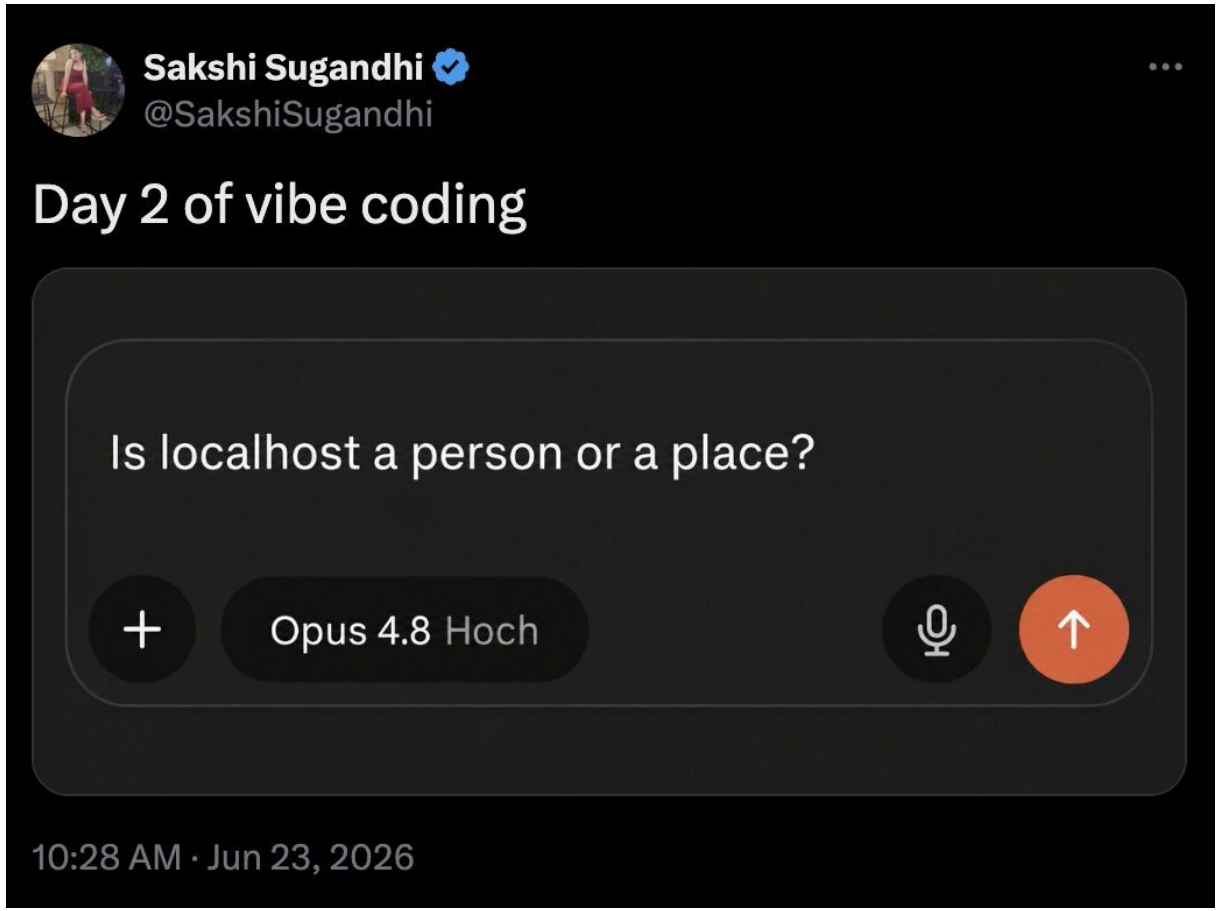
„Also bei mir läuft's.“

- Bei euch: localhost, eure Dateien, eure .env, euer Cache
- Live: anderer Rechner, andere Pfade, HTTPS, Groß-/Kleinschreibung
- Heute lernt ihr, wo die Unterschiede stecken

© 2026 TAISC – The AI Software Company GmbH

LOCALHOST

Day 2 of vibe coding



© 2026 TAISC – The AI Software Company GmbH

DREI UMGEBUNGEN

Wo läuft eure Seite eigentlich?

DATEI

`file:///Users/ben/Downloads/index.html`

Nur du

Kein Server. Kein fetch auf eigene Dateien, keine URL zum Teilen.

→

LOKALER SERVER

`http://localhost:5173`

Nur dein Rechner

Echter Server, echte Requests. „Läuft bei mir" heißt meistens: hier.

→

HOSTING

`https://team.github.io/projekt`

Alle, weltweit

Andere Umgebung, andere Pfade, HTTPS, Caching. Hier zeigt sich, ob es wirklich läuft.

© 2026 TAISC – The AI Software Company GmbH

DREI ADRESSLEISTEN

Was wo funktioniert

`file:///Users/lea/projekt/index.html`

- ✓ HTML und CSS öffnen
- ✗ fetch() auf lokale JSON-Dateien
- ✗ <script type="module"> (CORS)
- ✗ Pfade wie /img/… zeigen auf die Festplatte

`http://localhost:5173/`

- ✓ echter Server, echte Requests
- ✓ Module, fetch, Hot Reload
- ! nur auf eurem Rechner erreichbar
- ! eure .env ist hier, live nicht

`https://team.github.io/projekt/`

- ✓ öffentlich, HTTPS, CDN
- ! liegt in Unterordner /projekt/
- ! Groß-/Kleinschreibung zählt
- ✗ kein eigener Server-Code

Deshalb immer mit Dev-Server arbeiten (npm run dev) und nicht per Doppelklick auf die HTML-Datei.

© 2026 TAISC – The AI Software Company GmbH

TYPISCHE FALLEN

Warum es lokal läuft und live nicht

Pfade

- /img/logo.png zeigt auf die Domain-Wurzel
- Bei GitHub Pages liegt das Projekt in /repo/
- Logo.png ≠ logo.png auf dem Server

Umgebung

- API-Keys stehen nur in eurer lokalen .env
- Andere Node-Version, andere Pakete
- Datenbank nur auf eurem Rechner

Browser-Regeln

- HTTPS-Seite lädt kein http://
- CORS blockiert fremde APIs
- Cache zeigt die alte Version

© 2026 TAISC – The AI Software Company GmbH

PFADE IN BILDERN

Warum das Logo live fehlt

Seite: https://team.github.io/projekt/index.html

`src="img/logo.svg"`	→	`team.github.io/projekt/img/logo.svg`	200
`src="./img/logo.svg"`	→	`team.github.io/projekt/img/logo.svg`	200
`src="/img/logo.svg"`	→	`team.github.io/img/logo.svg` ← /projekt fehlt	404
`src="img/Logo.svg"`	→	…/projekt/img/Logo.svg ← Datei heißt logo.svg	404

Lokal funktionieren oft alle vier: Der Vite-Server liegt auf der Wurzel, und macOS/Windows ignorieren Groß-/Kleinschreibung.

© 2026 TAISC – The AI Software Company GmbH

DEPLOYEN

Euer Projekt auf GitHub Pages

Kostenlos, ohne Server-Kenntnisse – für statische Seiten genau richtig.

Repo auf GitHub

1 public

index.html liegt im Hauptordner.

Settings → Pages

2 1 Klick

Source: Deploy from a branch, Branch main, Ordner /root.

Warten

3 ca. 1 Min.

GitHub baut und veröffentlicht die Seite.

Live

4 URL

team.github.io/projekt – in die README damit.

Statisch reicht: HTML, CSS, JavaScript. Die APIs ruft der Browser direkt auf.

© 2026 TAISC – The AI Software Company GmbH

MIT VITE DEPLOYEN

Vite-Projekt auf GitHub Pages

1 · vite.config.js

```
import { defineConfig } from "vite";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  // Repo-Name mit Schrägstrichen
  base: "/projekt/",
  plugins: [tailwindcss()],
});
```

2 · Settings → Pages → Source:
„GitHub Actions“ wählen

3 · .github/workflows/deploy.yml (gekürzt)

```
on:
  push:
    branches: ["main"]
permissions:
  pages: write
  id-token: write
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v7
      - run: npm ci
      - run: npm run build
      - uses: actions/upload-pages-artifact@v5
        with: { path: ./dist }
      - uses: actions/deploy-pages@v5
```

Ohne Build reicht „Deploy from a branch“. Mit Vite muss GitHub erst bauen – und nur dist/ ausliefern.

© 2026 TAISC – The AI Software Company GmbH

QUELLORDNER ODER DIST?

Was genau online gehen muss

Quellordner – was ihr schreibt

```
projekt/
├── index.html   ← lädt /src/main.js
├── src/
│   ├── main.js  ← import "./style.css"
│   └── style.css ← @import "tailwindcss"
├── node_modules/ (Tausende Dateien)
└── package.json
```

Im Browser direkt: versteht `import` aus npm nicht, kennt `@import "tailwindcss"` nicht → weiße oder ungestylte Seite

→ `npm run build` →

dist/ – was online gehört

```
dist/
├── index.html
└── assets/
    ├── index-BYMXB0qK.css  ← fertiges CSS
    └── index-DH6xUsbR.js   ← gebündeltes JS
```

Im Browser direkt: normales HTML, CSS und JavaScript – läuft überall, auch auf GitHub Pages

GitHub Actions führt npm run build für euch aus und veröffentlicht nur dist/. Vercel, Netlify und Cloudflare machen das ohne YAML.

© 2026 TAISC – The AI Software Company GmbH

HOSTING IM VERGLEICH

GitHub Pages, Vercel, Netlify, Cloudflare

	GitHub Pages	Vercel	Netlify	Cloudflare
Statische Seite	ja	ja	ja	ja
Backend-Code	nein	Functions (Node)	Functions (Node)	Workers (eigene Laufzeit)
Build mit Vite	per GitHub Action	automatisch	automatisch	automatisch
Gratis enthalten	100 GB Traffic, 1 GB Seite	100 GB Traffic, 1 Mio. Function-Aufrufe / Monat	300 Credits / Monat (ein Deploy = 15)	statisch unbegrenzt, 100.000 Worker-Aufrufe / Tag
Wenn das Limit erreicht ist	Hinweis von GitHub	Pause bis zum nächsten Monat	Seite pausiert bis Monatsende	Worker antwortet mit Fehler bis zum nächsten Tag
Kommerziell im Gratis-Plan	nicht für Shops oder Business	nein (nur privat)	ja	ja
Preview-Link pro Branch	nein	ja	ja	ja
Stärke	Code und Seite an einem Ort	Next.js-Heimat, sehr einfach	Formulare ohne Server	schnellstes Netz, großzügigstes Free

Stand Oktober 2026, laut den Preisseiten der Anbieter. Limits ändern sich – vor dem Einsatz in echten Projekten nachschauen.

© 2026 TAISC – The AI Software Company GmbH

FÜR EUER PROJEKT

Zwei Stufen – beide sind erlaubt

Stufe 1 · Pflicht

Statisch + öffentliche API

Browser
`team.github.io`

→

Öffentliche API
`pokeapi.co · open-meteo.com`

- ✓ GitHub Pages reicht, kostenlos
- ✓ API braucht **keinen** oder nur einen **öffentlichen** Key
- ✓ API erlaubt CORS
- × kein Platz für geheime Keys

Stufe 2 · Bonus

Statisch + eigene Serverless Function

Browser
`/api/wetter`

→

Function
hält den geheimen Key

→

Fremde API

- ✓ Key bleibt auf dem Server, nie im Browser
- ✓ kein CORS-Problem (Server fragt Server)
- ✓ Vercel, Netlify oder Cloudflare – Free-Plan reicht
- → das ist euer „eigener Webserver", ohne Server zu betreiben

Pflicht ist Stufe 1. Wer einen geheimen Key braucht oder den Bonus „eigener Server" will, nimmt Stufe 2.

© 2026 TAISC – The AI Software Company GmbH

EXKURS SERVERLESS

Ein Backend in einer Datei

api/wetter.js (Vercel)

```
// läuft auf dem Server, nicht im Browser
export default async function handler(req, res) {
  const stadt = req.query.stadt ?? "Karlsruhe";
  const url = `https://api.wetter.example/heute`
    + `?stadt=${stadt}`
    + `&key=${process.env.WETTER_KEY}`;

  const antwort = await fetch(url);
  const daten = await antwort.json();
  res.status(200).json(daten);
}
```

main.js (Frontend)

```
// gleiche Domain → kein CORS,
// kein Key im Browser
const res = await fetch(
  "/api/wetter?stadt=Karlsruhe");
const wetter = await res.json();

// WETTER_KEY steht nur in den
// Vercel-Einstellungen →
// Environment Variables
```

Datei in den Ordner api/ legen, Repo mit Vercel verbinden – fertig. Bei Netlify heißt der Ordner netlify/functions, bei Cloudflare schreibt man einen Worker.

© 2026 TAISC – The AI Software Company GmbH

UNTER DER HAUBE

Ein Webserver in 10 Zeilen

server.js – node server.js

```
import { createServer } from "node:http";

createServer((req, res) ⇒ {
  console.log(req.method, req.url);
  res.writeHead(200, {
    "Content-Type": "application/json"
  });
  res.end(JSON.stringify({ hallo: "DHBW" }));
}).listen(3000);
```

- Das macht jeder Webserver im Kern: Anfrage rein, Antwort raus
- Aufruf im Browser: http://localhost:3000
- Express, Next.js, Supabase – alles baut darauf auf
- Optional für euer Projekt, aber gut zu wissen

© 2026 TAISC – The AI Software Company GmbH

STATISCH ODER DYNAMISCH

Wo entsteht das HTML?

Statisch

Fertige Dateien

Server liefert HTML, CSS, JS, wie sie auf der Platte liegen.
Beispiel: euer Projekt auf GitHub Pages.
+ schnell, billig, sicher

Dynamisch (Server baut HTML)

Pro Anfrage neu

Server fragt die Datenbank und baut das HTML für genau diese Anfrage.
Beispiel: Moodle, Online-Shop mit Login.
+ personalisiert – braucht Server

Statisch + API

Seite holt Daten nach

Statische Seite, JavaScript lädt JSON von einer API.
Beispiel: tagesschau.de, euer Abfahrtsmonitor.
+ das Beste aus beidem

Euer Projekt ist statisch + API: GitHub Pages liefert die Dateien, JavaScript holt die Daten.

© 2026 TAISC – The AI Software Company GmbH

DATEN IM BROWSER

Wo eine Web-App etwas merken kann

localStorage

Text, bleibt bis jemand ihn löscht. Perfekt für Favoriten, letzte Suche, Theme. Etwa 5 MB pro Seite.

sessionStorage

Wie localStorage, aber weg, sobald der Tab zu ist.

Cookies

Klein, gehen bei jeder Anfrage an den Server mit. Deshalb für Logins, nicht für Daten.

Kein Tresor

Alles davon kann der Nutzer lesen und ändern. Passwörter und Keys haben dort nichts verloren.

Für euer Projekt reicht localStorage – und es macht die App sofort persönlicher.

© 2026 TAISC – The AI Software Company GmbH

IM VERGLEICH

localStorage, sessionStorage, Cookies

	localStorage	sessionStorage	Cookies
Geht mit an den Server?	nie	nie	bei jeder Anfrage automatisch
Wer kann schreiben?	nur JavaScript	nur JavaScript	JavaScript und der Server (Set-Cookie)
Vor JavaScript versteckbar?	nein	nein	ja, mit HttpOnly
Wie lange?	bis jemand löscht	bis der Tab zu ist	bis zum Ablaufdatum
Wie viel?	ca. 5 MB	ca. 5 MB	ca. 4 KB pro Cookie
Typisch für	Favoriten, Theme, Entwürfe	Formular-Zwischenstand	Login, Sitzung, Spracheinstellung

Merksatz: Cookies sind für die Unterhaltung mit dem Server, localStorage ist das Notizbuch des Browsers.

© 2026 TAISC – The AI Software Company GmbH

LOCALSTORAGE IN DER PRAXIS

Favoriten, die das Neuladen überleben

favoriten.js

```
const KEY = "favoriten";

export function laden() {
  const text = localStorage.getItem(KEY);
  return text ? JSON.parse(text) : [];
}

export function speichern(liste) {
  localStorage.setItem(KEY,
    JSON.stringify(liste));
}
```

Verwendung

```
const favs = laden();      // [] beim ersten Mal
favs.push("Kronenplatz");
speichern(favs);

// DevTools → Application →
// Local Storage → eure Domain:
// favoriten = ["Kronenplatz"]
```

localStorage speichert nur Text. Listen und Objekte deshalb mit JSON.stringify hinein und JSON.parse heraus.

© 2026 TAISC – The AI Software Company GmbH

SICHERHEIT

Drei Dinge, die ihr nicht falsch machen dürft

Keys nicht ins Frontend

Alles im Browser ist öffentlich

- Jeder kann den Quelltext lesen
- Ein Key im JavaScript ist ein veröffentlichter Key
- Braucht eine API einen geheimen Key: Server davor

Secrets nicht ins Repo

.env und .gitignore

- Einmal committet, bleibt es in der Historie
- GitHub warnt, Bots suchen aktiv danach
- Versehentlich gepusht? Key sofort neu erzeugen

Eingaben nie vertrauen

Auch denen aus dem Formular nicht

- Browser-Validierung ist Komfort, keine Sicherheit
- innerHTML mit fremden Daten öffnet XSS
- Prüfen muss immer der Server

© 2026 TAISC – The AI Software Company GmbH

VORSICHT MIT .ENV

Was im Frontend landet, ist öffentlich

.env (steht in .gitignore)

```
# landet im Browser-Bundle!
VITE_WETTER_URL=https://api.wetter.example

# bleibt auf eurem Rechner –
# aber Vite gibt es dem Browser
# gar nicht erst
GEHEIMER_KEY=sk_live_123…
```

main.js

```
// funktioniert – und steht danach
// in dist/assets/index-….js
const url = import.meta.env.VITE_WETTER_URL;

// undefined: kein VITE_-Präfix
const key = import.meta.env.GEHEIMER_KEY;
```

Alles mit VITE_-Präfix kann jeder im Quelltext lesen. Geheime Keys gehören hinter einen eigenen Server.

© 2026 TAISC – The AI Software Company GmbH

KEYS IN DER PRAXIS

Welcher Key darf ins Frontend?

Kein Key nötig

PokéAPI, Open-Meteo, Open Food Facts, REST Countries. Ideal für GitHub Pages.

Öffentlicher Key

Gedacht fürs Frontend, beim Anbieter auf eure Domain eingeschränkt (z. B. Karten-APIs, Supabase „anon key“ mit Row Level Security).

Geheimer Key

KI-APIs, Bezahl-APIs, alles ohne Domain-Einschränkung. Darf nie in den Browser – auch nicht mit VITE_-Präfix.

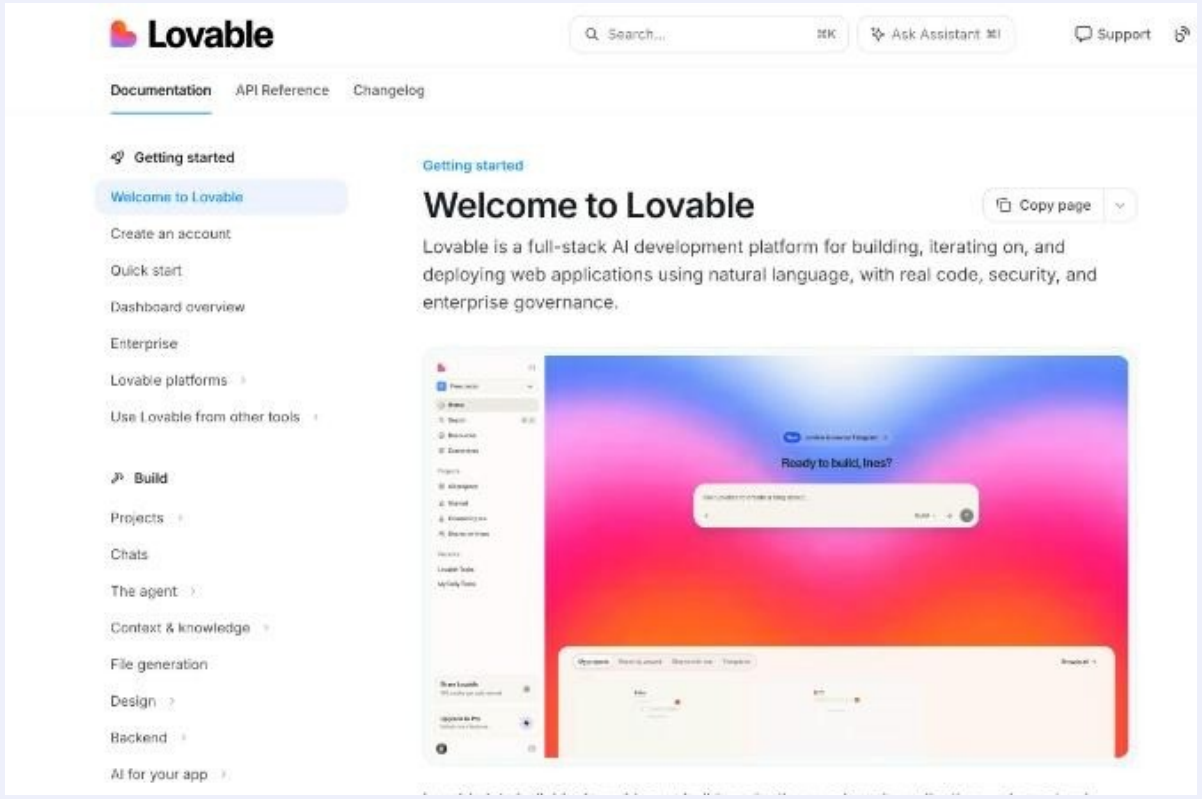
Dann: Stufe 2

Serverless Function auf Vercel, Netlify oder Cloudflare hält den Key. Das Frontend fragt nur eure Function.

© 2026 TAISC – The AI Software Company GmbH

LOVABLE, ARTIFACTS & CO.

Wo ist da der Server?



Prompt rein, eine Minute später ein Link

- Kein Server, kein Deploy, keine Datenbank – scheinbar
- Dahinter: Hosting, Datenbank, Login und Functions auf fremder Infrastruktur
- Bequem für Prototypen – aber Abhängigkeit, laufende Kosten, fremde Regeln

© 2026 TAISC – The AI Software Company GmbH

UNTER DER HAUBE

So funktioniert Lovable heute

1 · Ihr

Prompt im Chat

„Baue eine Abfahrts-App mit Login und Favoriten“

→

2 · KI-Agent

schreibt echten Code

React + Tailwind + shadcn/ui; neue Projekte seit Mai 2026 mit TanStack Start (Server-Rendering)

→

3a · Backend „Lovable Cloud“

Supabase unter der Haube

PostgreSQL · Login (Auth) · Dateien · Edge Functions · Secrets

3b · Hosting

Publish → name.lovable.app

HTTPS automatisch, weltweit ausgeliefert (CDN), eigene Domain im Bezahl-Plan

3c · Ausstieg

Git-Sync mit GitHub

Code gehört euch – auch eigenes Supabase-Projekt möglich

Alles, was wir heute besprochen haben, steckt drin: Build, Hosting, CDN, HTTPS, Datenbank, Secrets. Es ist nur versteckt.

© 2026 TAISC – The AI Software Company GmbH

MERKSATZ

Der Server ist nicht weg.
Er ist nur versteckt.

Wer weiß, was darunter passiert, kann entscheiden, wann das okay ist.

DREI WOCHEN BIS ZUM 17.11.

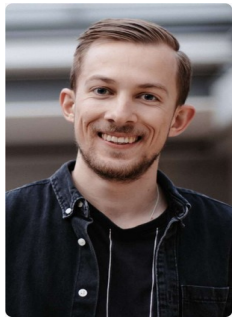
Euer Projekt geht online

Ziel bis zum nächsten Termin: responsive, deployed, mindestens drei Komponenten.
Details auf Übungsblatt 5 im Repo.

Danke.

Fragen?

Nächster Termin: 17.11. · JavaScript im Browser



Joshua Heller

Dozent Web-Engineering 1 · Gründer TAISC

M: dhbw@theaisoftwarecompany.com
L: linkedin.com/in/heller-joshua
W: theaisoftwarecompany.com/karriere